



WeCloudData

Planning and Task Decomposition

Introduction





WeCloudData

Planning and Task Decomposition



Why Planning Matters in AI Agents

The Problem With Reactive Agents

Early AI agents were purely reactive: receive input → produce output. This works for simple tasks but fails completely when a goal requires multiple coordinated steps.

Real-world example: Ask a reactive agent: "Research the top 5 competitors of Tesla, summarize their EV strategies, and produce a comparison table." A reactive agent either produces one shallow response or gets lost trying to do everything at once.

What a planning agent does instead:

- Understands the full goal before acting
- Breaks the goal into sub-tasks
- Sequences those sub-tasks logically
- Tracks which steps are done, pending, or failed
- Adjusts the plan when something goes wrong



WeCloudData

Task Decomposition



Task Decomposition

What Is Task Decomposition?

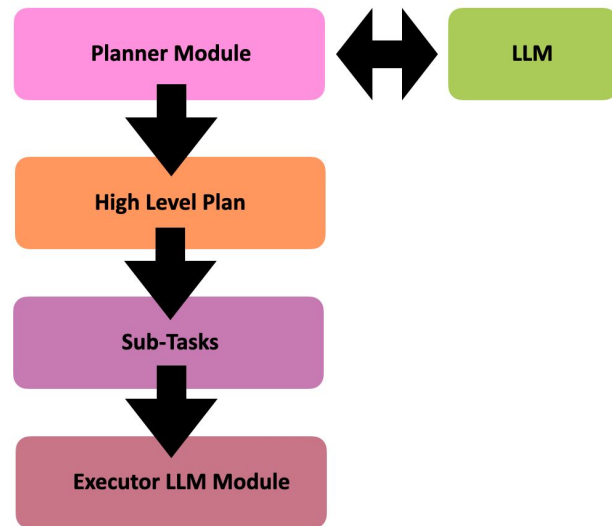
Task decomposition is the process of taking a high-level goal and breaking it into smaller, independently executable sub-tasks.

Why it's necessary:

- LLMs have limited working memory per prompt
- Complex goals span multiple tools, APIs, or reasoning steps
- Some steps depend on results from earlier steps
- Parallel execution becomes possible when tasks are separated

Decomposition produces:

- A list of concrete, bounded sub-tasks
- Dependencies between those sub-tasks
- An ordering or graph of execution





Task Decomposition

Three Decomposition Strategies

1. Sequential Decomposition Steps must happen in strict order. Each step depends on the output of the previous one. **Example:** Scrape webpage → parse HTML → extract entities → store in database

2. Parallel Decomposition Steps are independent and can run simultaneously. **Example:** Fetch weather data AND fetch news headlines AND fetch stock prices — then combine

3. Hierarchical Decomposition Tasks are broken into sub-tasks, which are themselves broken into sub-sub-tasks. A tree structure. **Example:** "Write a research report" → [Research phase, Writing phase, Review phase] → each phase has its own sub-steps

In practice: Most real agents use a combination - a hierarchical structure with sequential and parallel nodes at different levels.



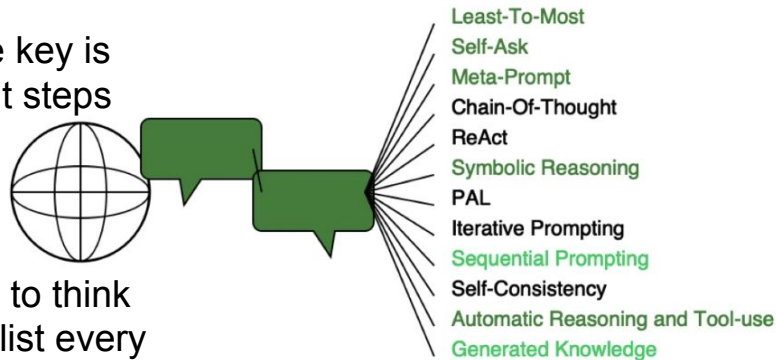
Task Decomposition

Decomposition as a Prompt Engineering Problem

LLMs can decompose tasks when prompted correctly. The key is structured prompting that forces the model to reason about steps before acting.

Common techniques:

1. **Chain-of-Thought (CoT) Prompting** Ask the model to think step-by-step before answering. "Before solving this, list every sub-task required."
2. **Plan-and-Execute Prompting** Separate the planning call from the execution calls. First prompt: generate a plan. Subsequent prompts: execute each step.
3. **ReAct Framework (Reason + Act)** The agent alternates between reasoning about what to do and taking an action, then observing the result. Format: **Thought** → **Action** → **Observation** → **Thought** → ...





Task Decomposition

Decomposition as a Prompt Engineering Problem

Structured Output Forcing Instruct the model to return a JSON task list, which the system then uses programmatically.

```
{
  "goal": "Research EV competitors",
  "tasks": [
    {"id": 1, "action": "search", "query": "Tesla competitors 2025"},
    {"id": 2, "action": "summarize", "depends_on": [1]},
    {"id": 3, "action": "format_table", "depends_on": [2]}
  ]
}
```




Planning Agents



Planning Agent

What Is a Planning Agent?

A planning agent is an AI agent that explicitly generates, maintains, and updates a plan before and during task execution. It does not simply react, it reasons about the future sequence of actions.

Core components of a planning agent:

Component	Role
Goal Parser	Understands the user's high-level intent
Planner Module	Generates a structured plan (task list + order)
Executor	Carries out individual steps using tools
State Tracker	Maintains which steps are done, pending, failed
Replanner	Updates the plan when execution reveals new information



Planning Agent

Modern Planning Agent Architectures in Practice

Plan-and-Execute (LangChain / LangGraph)

- Step 1: A dedicated "planner" LLM call creates a full task list
- Step 2: An "executor" LLM (or tool) completes each task
- Step 3: Results are fed back; replanning occurs if needed
- Used in: AutoGPT descendants, OpenAI's Assistants API with function calling



Planning Agent

Modern Planning Agent Architectures in Practice

Hierarchical Task Network (HTN)

- Classic AI planning formalized for LLM agents
- Tasks are decomposed recursively until "primitive" (directly executable) tasks are reached
- Good for structured domains (logistics, coding assistants)



Planning Agent

Modern Planning Agent Architectures in Practice

Graph-Based Planning (LangGraph)

- The plan is represented as a directed graph (nodes = steps, edges = dependencies)
- Supports branching, looping with conditions, and parallel execution
- State is maintained across the entire graph



Planning Agent

Modern Planning Agent Architectures in Practice

Multi-Agent Planning

- Different agents own different parts of the plan
- An orchestrator agent delegates sub-plans to specialist agents
- 2025 standard: used in frameworks like CrewAI, AutoGen, and Google's Agent Space



Demo: Task Planning Agent





Step Generation



Step Generation

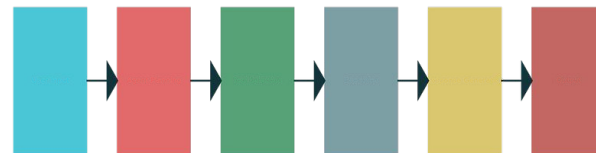
Turning Goals into Actions

Step generation is the process of converting a decomposed task list into concrete, executable instructions that an agent's tools can act on.

The gap between decomposition and execution: Decomposition says: "Search for competitor information" Step generation says: "Call the web_search tool with query='Tesla EV competitors market share 2025', return top 5 results, extract company names and key metrics"
Agentic AI Workflow

Good step generation produces steps that are:

- **Atomic** - one action per step, not bundled
- **Tool-specific** - mapped to an actual available tool or API
- **Parameterized** - includes all inputs the tool needs
- **Verifiable** - has a clear success/failure condition
- **Scoped** - produces a bounded output (not "search everything")





Step Generation

Step Generation Techniques

Technique 1: Template-Based Generation Pre-defined step templates matched to task types. Fast and consistent but inflexible.

```
SEARCH_STEP = {  
    "tool": "web_search",  
    "query": "{user_query}",  
    "max_results": {n}  
}
```

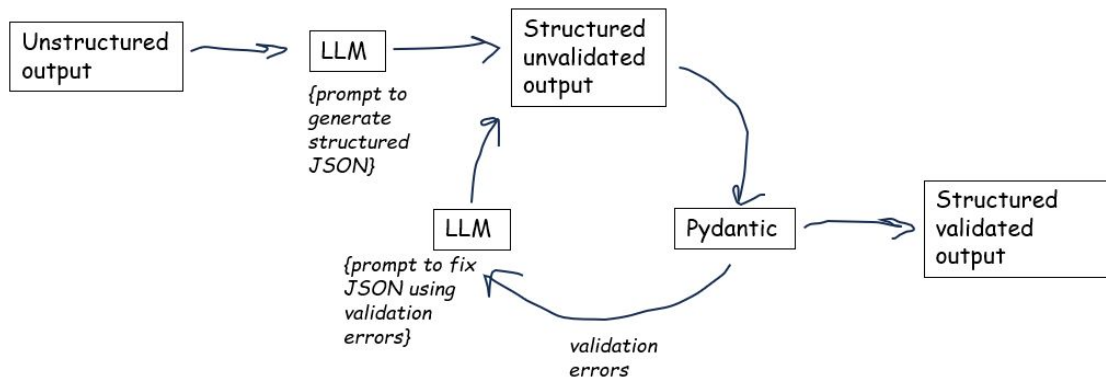


Step Generation

Step Generation Techniques

Technique 2: LLM-Generated Steps with Schema Enforcement

The LLM generates steps, but must conform to a JSON schema that maps directly to available tools. Pydantic or JSON Schema validation rejects malformed steps.





Step Generation

Step Generation Techniques

Technique 3: Tool-Aware Prompting The planner is given the full list of available tools (name, description, parameters) and generates steps that only reference real tools.

```
Available tools: [web_search, read_file, write_file, python_exec, send_email]
```

```
Goal: Summarize Q3 report and email it to the team
```

```
→ Steps:
```

1. `read_file(path="q3_report.pdf")`
2. `python_exec(code="summarize(content)")`
3. `send_email(to="team@co.com", body="{summary}")`



Step Generation

Step Generation Techniques

Technique 4: Dynamic Step Insertion

Mid-execution, if a step reveals an unexpected result, new steps are inserted into the plan before continuing.



Step Generation

Step Dependencies and Ordering

Steps often produce outputs that become inputs to later steps. If ordering is wrong, the agent tries to use data that doesn't exist yet.

Dependency types:

- **Data dependency:** Step B needs the output of Step A Fetch user data → Analyze user data (can't analyze before fetching)
- **Resource dependency:** Step B needs Step A to set up a resource Create database table → Insert records
- **Logical dependency:** Step B only makes sense after Step A Draft email → Proofread email → Send email
- **Representing dependencies:** Most modern planning agents use a **Directed Acyclic Graph (DAG)**:
 1. Nodes = steps
 2. Edges = "must complete before" relationships
 3. Execution engine processes nodes in topological order
 4. Parallel nodes (no dependency on each other) can execute concurrently



Execution Tracking



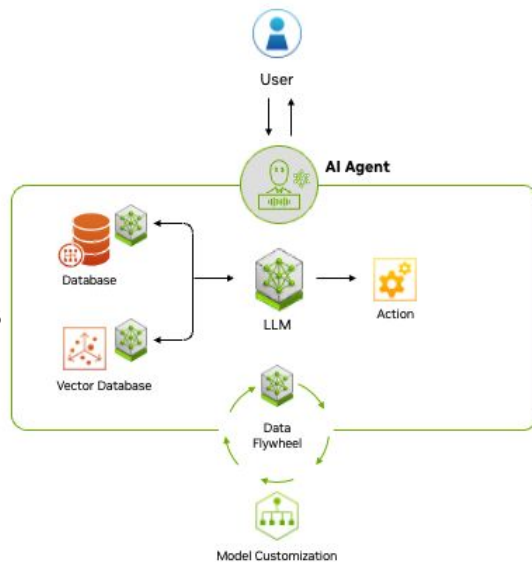
Execution Tracking

What Is Execution Tracking?

Execution tracking is the mechanism by which a planning agent maintains an accurate, real-time record of the state of every step in its plan.

Why it's critical:

- LLMs are stateless , they have no memory between calls
- Complex plans can have 10–50+ steps; tracking by memory is unreliable
- Steps can fail, time out, or produce unexpected output
- Agents must be able to resume, retry, or replan
- Without tracking, agents repeat work or miss steps entirely





Execution Tracking

What Is Execution Tracking?

What a tracker must record for each step:

Field	Values
Step ID	Unique identifier
Status	pending / in-progress / complete / failed / skipped
Input	What was passed to this step
Output	What was returned
Timestamp	When it started and ended
Error	Error message if failed
Retry count	How many times it's been attempted



Execution Tracking

Implementation Patterns

How Tracking Is Built

Pattern 1: In-Memory State Dictionary Simple Python dict or Pydantic model. Works for short-lived agents. Lost if process crashes.

```
state = {  
    "step_1": {"status": "complete", "output": "..."},  
    "step_2": {"status": "in-progress", "output": None},  
    "step_3": {"status": "pending", "output": None}  
}
```



Execution Tracking

Implementation Patterns

Pattern 2: Persistent State Store State is written to a database (SQLite, Redis, PostgreSQL) after every step. Agent can resume from last checkpoint if interrupted.

Pattern 3: LangGraph State Management LangGraph treats the entire graph execution as a stateful object. Each node receives the full state and returns an updated state. Built-in checkpointing is available.

Pattern 4: Event Sourcing Every state transition is logged as an immutable event. The current state is derived by replaying events. Excellent for debugging and auditing.

For production agents, always use persistent state with checkpointing. The cost of replaying from scratch after a crash is too high for long-running tasks.



Demo: Execution Tracking





WeCloudData

Preventing Infinite Loops



Preventing Infinite Loops

Handling Failures in Execution

A robust planning agent must have explicit policies for every failure type.

Failure types and responses:

Failure Type	Example	Response Strategy
Tool error	API returns 500	Retry with backoff
Timeout	Step takes >30s	Kill and retry or skip
Bad output	LLM returns malformed JSON	Re-prompt or use fallback parser
Dependency failure	Step 2 fails, Step 3 needs it	Mark Step 3 as "skipped"
Logical failure	Output contradicts goal	Trigger replanning



Preventing Infinite Loops

Handling Failures in Execution

Retry strategies:

- **Immediate retry** - Try again instantly (good for transient errors)
- **Exponential backoff** - Wait 1s, 2s, 4s, 8s between retries (good for rate limits)
- **Alternative path** - Use a different tool or approach entirely
- **Human-in-the-loop** - Pause and ask for user input when stuck



Preventing Infinite Loops

Why Agents Get Stuck in Loops

Infinite loops are one of the most dangerous failure modes in agentic systems. They waste resources, can incur significant API costs, and produce no useful output.

How loops occur:

- **Loop Type 1: Replanning Loop** Agent executes Step 1 → Step 1 fails → agent replans → new plan includes Step 1 → fails again → replans again... forever.
- **Loop Type 2: Tool Call Loop** Agent calls a tool → receives an ambiguous result → calls the same tool again with a slightly different query → same ambiguous result → calls again...
- **Loop Type 3: Reasoning Loop** Agent's reasoning trace keeps returning to the same "Thought" without making progress. Common in poorly prompted ReAct agents.
- **Loop Type 4: Dependency Loop (Circular)** Step A depends on Step B, which depends on Step A. If not caught at plan-time, this hangs execution permanently.

Real cost: A looping GPT-4o agent making tool calls can consume hundreds of dollars in API costs per hour. Loop prevention is not optional, it is a safety requirement.



Preventing Infinite Loops

Loop Detection Strategies

How to Detect Loops Before They're Expensive

Strategy 1: Maximum Step Count Hard limit on the total number of steps an agent can execute in one session.

```
MAX_STEPS = 25
if execution_count >= MAX_STEPS:
    raise AgentLoopError("Step limit exceeded")
```



Preventing Infinite Loops

Loop Detection Strategies

Strategy 2: Step Repeat Detection Track a hash of (step_type + input_parameters). If the same hash appears twice, the agent is looping.

```
seen_steps = set()
step_hash = hash((step.tool, str(step.params)))
if step_hash in seen_steps:
    raise AgentLoopError("Duplicate step detected")
seen_steps.add(step_hash)
```



Preventing Infinite Loops

Loop Detection Strategies

Strategy 3: Progress Assertion

After every N steps, verify that meaningful progress has been made toward the goal. If state hasn't changed, terminate.

Strategy 4: DAG Validation at Plan Time

Before execution begins, run a cycle detection algorithm (topological sort) on the dependency graph. If a cycle exists, reject the plan.

Strategy 5: Time-Based Budget

Set a wall-clock time limit. If the agent hasn't completed within that time, terminate.



Preventing Infinite Loops

Loop Prevention: Design Principles

Building Loop-Safe Agents by Design

Loop prevention should be baked into the architecture, not bolted on as an afterthought.

Principle 1: Finite Plans Only

A plan must have a defined end state. Open-ended plans ("keep searching until you find the perfect answer") must be converted to bounded ones ("search up to 10 sources, then synthesize").

Principle 2: Monotonic Progress

Every step must move the agent *forward*. If a step can produce a state identical to a previous state, it must include a counter or guard preventing repetition.



Preventing Infinite Loops

Loop Prevention: Design Principles

Principle 3: Replanning Limits

Allow replanning, but cap it. An agent should not replan more than 2–3 times for a single goal. Each replan must produce a different plan, not the same one.

Principle 4: Explicit Termination Conditions

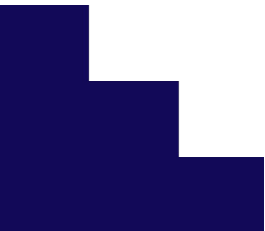
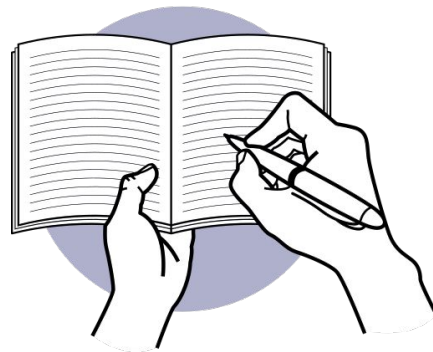
Define exactly what "done" looks like before execution begins. "Done when: all 5 competitor summaries are written AND the comparison table is populated AND the output file is saved."

Principle 5: Human Escalation

When an agent detects it is looping, escalate to a human rather than retrying indefinitely. This is the correct behavior for production systems.



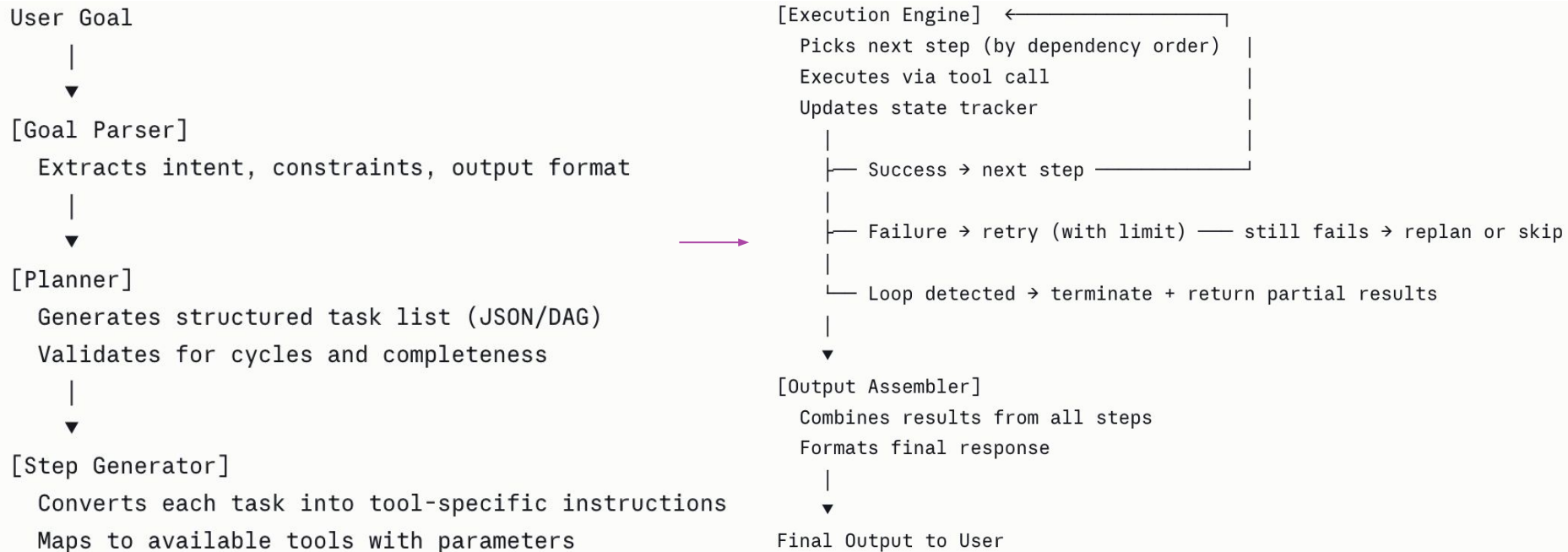
Exercise — Loop Prevention





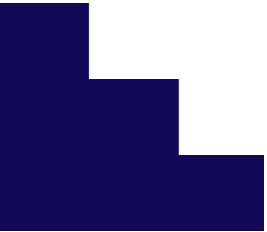
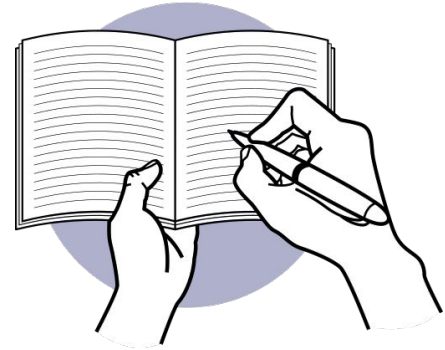
Full Agent Architecture

Putting It All Together





Exercise — Implement a Planning Agent





WeCloudData

Thank You

We'd love to hear from you. If you have any questions,
feel free to reach out to our team anytime!